

Learned Page Replacement under Workload Shift: FIFO, LRU, Optimal and a Decision-Tree Policy

CSE-307: Operating Systems, Part-B
Md Miraz Morshed
202414065
CSE 24, MIST

03 October 2026

Abstract

Classical page replacement policies such as FIFO and LRU assume a reasonably stable access pattern. I implement FIFO, LRU and Belady’s Optimal policy and a lightweight decision-tree eviction policy (static and periodically retrained) and evaluate all of them on a synthetic trace whose pattern shifts from locality-heavy sequential access to random bursty access halfway through. The learned policy helps when the classical policies thrash on a loop larger than memory, but it provides no benefit after the shift, where all non-optimal policies degrade by roughly the same amount.

1 Problem Framing

Demand paging relies on a replacement policy to choose which resident page to evict on a fault when no free frame exists. Belady’s Optimal policy evicts the page whose next use is farthest in the future; it minimises faults but needs future knowledge, so it is only a benchmark [1]. FIFO is simple but ignores usage and LRU approximates Optimal by assuming the recent past predicts the near future [3]. Both rely on locality of reference, which real programs exhibit only within phases; at phase boundaries the working set changes [2].

This paper asks: (i) can a small model using recency and frequency features learn a better eviction rule than FIFO/LRU and (ii) how do the classical and learned policies behave when the access pattern shifts mid-run?

2 Implementation Summary

All code is in Python (repository: github.com/miraz3301). Each policy exposes `simulate(trace, frames)` and returns a hit/fault flag per access. Each policy exposes `simulate(trace, frames)` and returns a hit/fault flag per access.

Classical policies. FIFO uses a queue of loaded pages. LRU uses an ordered dictionary updated on every hit. Optimal precomputes, for every position, the index of the next access to the same page and evicts the resident page with the largest next-use index.

Learned policy. On a fault with full memory, each resident page is described by four features computable online: recency (time since last access), frequency (accesses since load), age (time since load) and access rate (frequency/age). A depth-5 decision tree (scikit-learn [7]) outputs the probability that the page is a “good eviction candidate” and the page with the highest score is evicted (ties broken by older last access). Training labels come from hindsight: Belady’s policy is replayed on a training trace and at every eviction each resident page is labelled 1 if its next use is

at or beyond the median next-use distance of the resident set. Using Belady’s decisions as a teacher follows the idea of learning from the optimal policy [5]. Two variants are evaluated:

- **Learned (static):** trained once on a pre-shift trace generated with a different random seed, so it never sees the post-shift pattern.
- **Learned (adaptive):** additionally refit every 2000 accesses on the window just observed, with labels computed by hindsight inside that window.

3 Experimental Setup

Workload. Each trace has 20,000 accesses over 200 distinct pages. The first 10,000 accesses are locality-heavy: repeated sequential scans of a 48-page region (drifting slowly), interleaved with frequent touches (probability 0.4) of an 8-page hot set. The last 10,000 accesses are random and bursty: uniform random pages, with 30% of picks repeated 2–6 times in a burst. Resident pages carry over across the shift; only the pattern changes.

Parameters. Memory sizes of 32, 64 and 128 frames; five random seeds per setting; results are means over seeds (raw data in `results/results.csv`). “Before” and “after” refer to the first and second half of the trace. Cold-start faults (about 188 distinct pages touched) are included in “before”.

4 Results

Table 1 gives the mean hit ratio before and after the shift; Table 2 gives page faults per half (each half has 10,000 accesses). Figures 1 and 2 summarise the same data.

Table 1: Mean hit ratio, before → after the shift.

Policy	32 frames	64 frames	128 frames
FIFO	0.166 → 0.554	0.979 → 0.639	0.980 → 0.808
LRU	0.211 → 0.554	0.981 → 0.639	0.981 → 0.808
Learned (static)	0.311 → 0.554	0.981 → 0.639	0.981 → 0.809
Learned (adaptive)	0.327 → 0.556	0.981 → 0.637	0.981 → 0.807
Optimal	0.648 → 0.722	0.981 → 0.820	0.981 → 0.934

Table 2: Mean page faults, before → after the shift.

Policy	32 frames	64 frames	128 frames
FIFO	8341 → 4457	205 → 3607	196 → 1917
LRU	7889 → 4458	188 → 3608	188 → 1919
Learned (static)	6894 → 4461	188 → 3608	188 → 1908
Learned (adaptive)	6730 → 4445	188 → 3629	188 → 1926
Optimal	3524 → 2783	188 → 1804	188 → 658

5 Analysis: What Changed Under the Shift

Which policy degrades most? With 64 and 128 frames the whole pre-shift working set (the 48-page loop plus 8 hot pages) fits in memory, so every policy, including the learned ones, reaches about 0.98 and faults only on cold start. After the shift, hit ratio falls to about 0.64 (64 frames) and 0.81 (128 frames) for FIFO, LRU and both learned policies, a drop of roughly 0.34 and 0.17, while

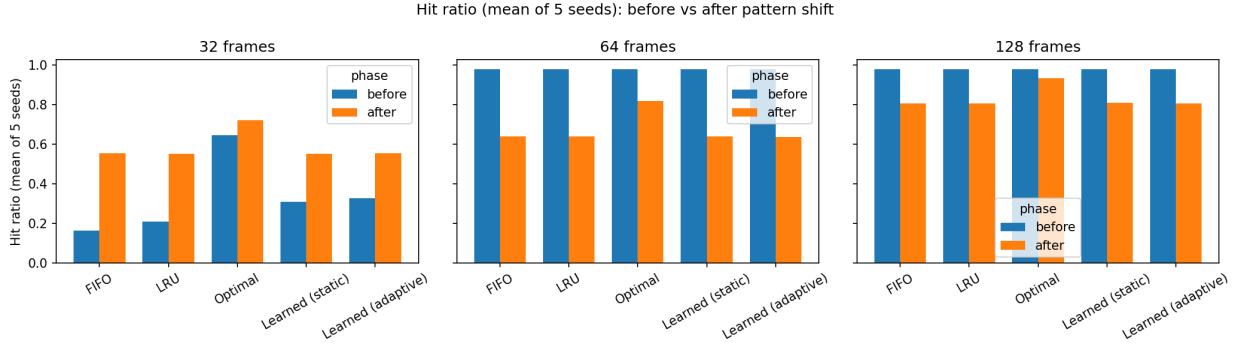


Figure 1: Hit ratio before and after the shift (mean of 5 seeds).

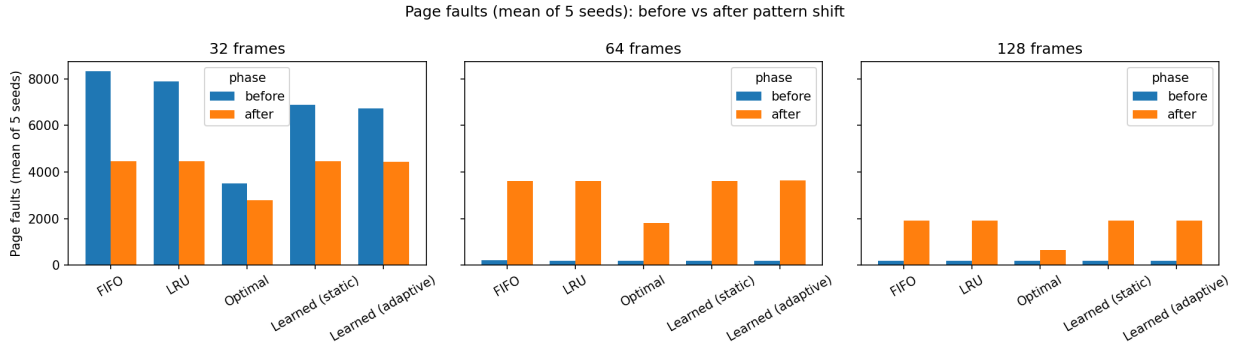


Figure 2: Page faults before and after the shift (mean of 5 seeds).

Optimal falls only to 0.82 and 0.93. Thus no non-optimal policy is meaningfully more fragile than the others; they all lose the same locality. FIFO and LRU make decisions from past order or recency, which carry little information about a uniform random future. A uniformly random page is resident with probability about m/N for m frames and $N = 200$ pages, regardless of which resident pages are kept, plus the free hits inside bursts. About 47% of post-shift accesses are repeats within a burst, so the expected hit ratio is roughly $0.47 + 0.53m/N$, i.e. 0.64 for $m = 64$ and 0.81 for $m = 128$, matching Table 1. Only Optimal, which sees the future, can avoid evicting soon-to-be-used pages.

Where learning helped. With 32 frames the 48-page loop exceeds memory and LRU and FIFO thrash on it (hit ratio 0.21 and 0.17). The tree, which relies mainly on recency (feature importance 0.72 at 32 frames) with frequency and access rate as secondary signals, reaches 0.31 (static) and 0.33 (adaptive). This is a pre-shift gain; after the shift all non-optimal policies are tied again. At 32 frames the hit ratio rises after the shift only because the pre-shift pattern was pathological for small memory.

Does adapting help? Retraining every 2000 accesses gave a small gain before the shift at 32 frames but no measurable benefit after the shift. Post-shift, the random pattern has no learnable structure in recency or frequency, so even a model retrained on the new data cannot approach Optimal. Differences of a few faults between the learned variants and LRU after the shift (Table 2) are within seed-to-seed noise (mean differences between -14 and $+20$ faults, against standard deviations of 14–29 across seeds).

Limitations. The trace is synthetic, with only one shift and one hindsight-labelling rule; features are limited to four per-page statistics; results are means over five seeds and the model is retrained from scratch rather than updated incrementally. The inference cost of a tree per fault is far higher than LRU’s bookkeeping, so a real kernel would need a cheaper rule or hardware support. Adaptive policies such as ARC [4] and richer learned prefetchers [6] point to more promising directions.

6 Conclusion

A small decision tree trained on Belady’s decisions can beat FIFO and LRU when the pattern is stable but poorly matched to classical heuristics, but when the workload shifts to random access every non-clairvoyant policy loses locality in the same way and the learned policy offers no advantage. Learned components therefore need to be evaluated under distribution shift, not only on the pattern they were trained on.

AI Assistance Disclosure

ChatGPT was used to help write the simulator code, the workload generator and a first draft of this report. I ran the experiments, checked the numbers against the raw results and revised the analysis.

References

- [1] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [2] P. J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.
- [3] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*. Wiley (latest edition).
- [4] N. Megiddo, D. S. Modha. ARC: A self-tuning, low overhead replacement cache. In *USENIX FAST*, 2003.
- [5] A. Jain, C. Lin. Back to the future: Leveraging Belady’s algorithm for improved cache replacement. In *ISCA*, 2016.
- [6] M. Hashemi et al. Learning memory access patterns. In *ICML*, 2018.
- [7] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.